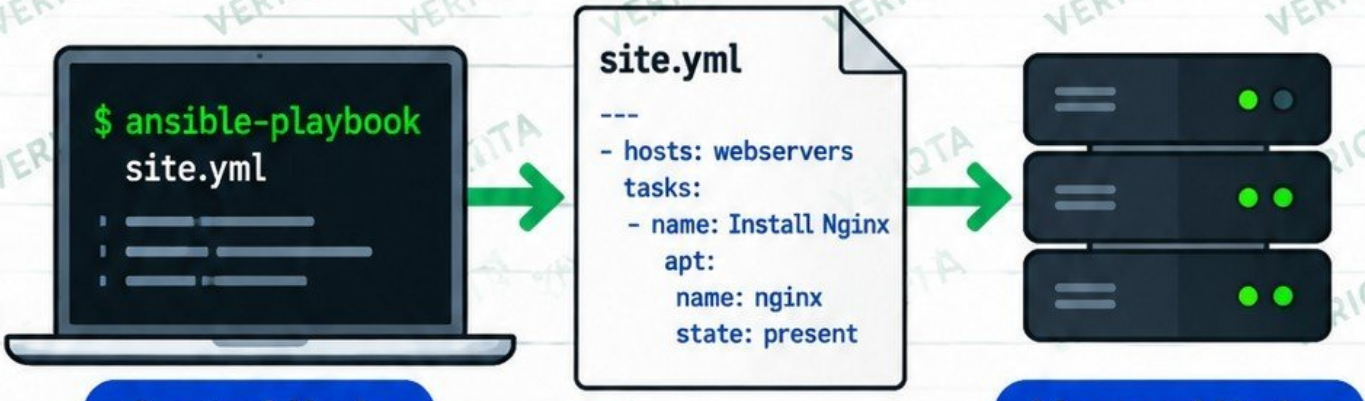
VERIQTA

Ansible for Beginners:

Your First Playbook



Control Node
Runs your playbook

Managed Nodes
Configures your servers

AUTOMATE CONFIGURATION

MANAGE SERVERS

WRITE PLAYBOOKS

BUILD REAL SKILLS

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

Ansible for Beginners: What Is Ansible, and Why Do DevOps Engineers Use It?

1. What Is Ansible?

- Ansible is an open-source automation tool used for configuration management, application deployment, and orchestration.
- It helps you configure and manage multiple servers using simple YAML playbooks.
- Ansible connects to servers over SSH and does not require an agent to be installed on the target servers.

2. What Is Configuration Management?

- Configuration management means keeping your servers in a desired state.
- You define what the server should look like (software, configuration, services, users).
- A tool like Ansible makes sure the server matches that state, every time.

Desired State

You describe the end result, and Ansible makes the changes needed to reach it.

3. The Problem With Manual Server Setup

- You have to run the same commands on many servers.
- It is slow and time-consuming.
- You may make mistakes.
- Servers can become inconsistent.
- It is difficult to track what changed.
- Repeating tasks for updates and new servers is hard to manage.

Real World Example

- Your company has 10 web servers.
- You need to install Nginx, create a user, and configure the firewall.
- If you do this manually, you must SSH into each server and run the same commands.
- This is slow, error-prone, and difficult to repeat when you add more servers.

4. Manual SSH Commands vs Ansible Playbook

Manual SSH Commands (Do this on each server)



```
ssh ubuntu@192.168.1.10
sudo apt update
sudo apt install -y nginx
sudo systemctl start nginx
sudo systemctl enable nginx
```

- Run these commands on every server.
- Takes time and can be inconsistent.
- Easy to make mistakes.
- Hard to repeat and track changes.

Ansible Playbook (Run once for all servers)



```
- name: Install and start Nginx
  hosts: webservers
  become: true
  tasks:
    - name: Install Nginx
      apt:
        name: nginx
        state: present
    - name: Start and enable Nginx
      systemd_service:
        name: nginx
        state: started
        enabled: true
```

- Define the desired state in a YAML playbook.
- Run once and it applies to all target servers.
- Repeatable, consistent, and easier to manage.
- You can safely run it many times.

5. Why DevOps Engineers Use Ansible

AUTOMATE	MANAGE	SAVE TIME	REDUCE ERRORS	SCALE	KEEP RECORDS
Repeatable infrastructure tasks	Multiple servers consistently	Eliminate manual repetitive work	Use tested playbooks	Handle many servers easily	Version control playbooks (Git)

Ansible for Beginners: How Ansible Works: Controller, Inventory, and Managed Nodes

1. Key Components of Ansible

Control Node
- The machine where you run Ansible commands.
- Has Ansible installed.
- Reads inventory and playbooks.

Inventory
- A file that lists your target servers.
- Organizes servers into groups.
- Can use IP addresses or hostnames.

Managed Nodes
- The remote servers you want to configure.
- Usually Linux servers.
- Do not need Ansible installed.

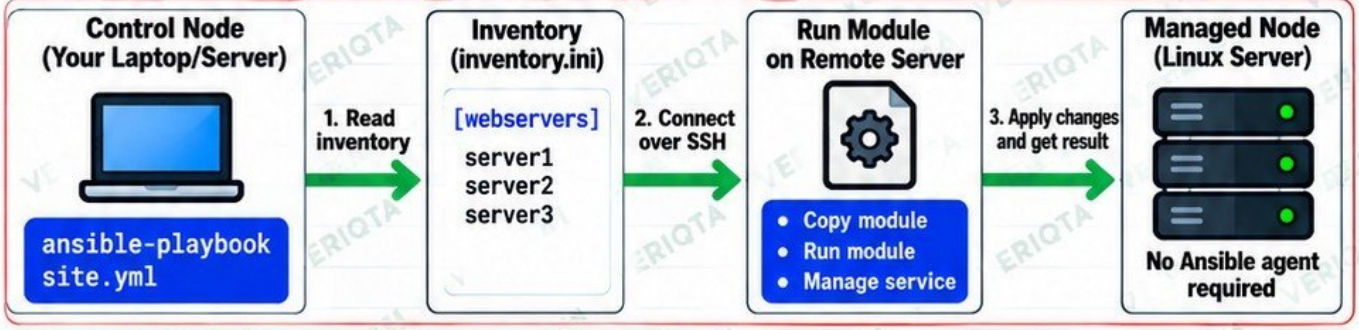
Modules
- Small, reusable pieces of code.
- Perform specific tasks (install packages, manage files, start services, etc).

Playbooks
- YAML files that define what to do.
- Use modules to perform tasks.
- Can run on one or many servers.

2. How Ansible Works

1. You run an Ansible command or playbook on the control node.
2. Ansible reads the inventory to find the target servers.
3. Ansible connects to each managed node over SSH.
4. Ansible copies a small module file to the managed node.
5. The module runs on the remote server (e.g., install a package, manage a service).
6. The server returns the result (ok, changed, failed, or skipped).
7. Ansible shows a summary (play recap).

3. Execution Path: From Your Command to the Server



4. Example Command and What Happens

```
ansible-playbook -i inventory.ini site.yml
```

1. Ansible reads inventory.ini
2. Connects to each server over SSH
3. Runs the tasks defined in site.yml
4. Shows the result for each server

Example Play Recap

```
PLAY RECAP *********************************
server1    : ok=4   changed=2   unreachable=0   failed=0
server2    : ok=4   changed=2   unreachable=0   failed=0
server3    : ok=4   changed=0   unreachable=0   failed=0
```

5. Why Ansible Does Not Need an Agent

- Uses SSH to connect to the server.
- Copies small temporary module files when needed.
- Runs the module and removes it after execution.
- Only requires Python on the managed node (usually preinstalled).
- No long-running background agent is needed.
- This makes Ansible simple, lightweight, and easy to adopt.

6. Real World Use Cases

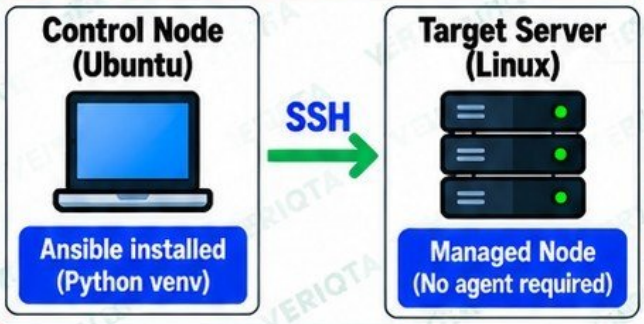
- Install and configure web servers.
- Deploy applications.
- Manage users and permissions.
- Update multiple servers at the same time.
- Keep server configurations consistent.
- Automate infrastructure tasks in cloud or on-premises.

Ansible for Beginners: Build Your First Ansible Lab

1. What We Are Building

- A Linux control environment (Ubuntu).
- Install Ansible in a Python virtual environment.
- Connect to a target server (another Linux server).
- Run your first Ansible command and playbook.
- A safe practice lab you can use for learning without touching production servers.

2. Lab Architecture



3. Prepare Your Control Node (Ubuntu)

1. Update your system

```
sudo apt update
sudo apt install -y python3 python3-venv
```

2. Create a project directory

```
mkdir -p ~/ansible-lab
cd ~/ansible-lab
```

3. Create and activate a Python virtual environment

```
python3 -m venv .venv
source .venv/bin/activate
```

4. Install Ansible inside the virtual environment

```
pip install --upgrade pip
pip install ansible
```

5. Verify the installation

```
ansible --version
```

4. Verify Ansible Installation

```
$ ansible --version
ansible [core 2.16.4]
  config file = None
  configured module search path = [...]
  python version = 3.11.6 (main, ... )
  executable location = .../.venv/bin/ansible
```

5. Control Node vs Target Server

Control Node (Your Laptop/VM)

- Runs Ansible commands and playbooks.
- Has Ansible installed.
- Reads the inventory file.
- Connects to target servers over SSH.
- Does not need to be the same server as the targets.

Target Server (Managed Node)

- The server you want to configure and manage.
- Usually a Linux server (Ubuntu/CentOS/etc).
- Does not need Ansible installed.
- Ansible uses SSH to connect and run modules.
- Can be one server or many servers.

6. Check Python and Ansible Paths

```
$ which python3
 /home/ubuntu/ansible-lab/.venv/bin/python3
$ which ansible
 /home/ubuntu/ansible-lab/.venv/bin/ansible
```

- This confirms Ansible is installed in your virtual environment.
- Your system Python is not affected.
- This keeps your environment clean and safe.

7. Safe Practice Environment

- Use a personal VM or cloud instance for learning.
- Do not use production servers.
- You can use a second Ubuntu VM as your target server.
- Practice installing packages, creating users, and configuring services.
- Make mistakes, test, and learn.
- Once you are comfortable, you can apply the same skills to real environments with proper approvals.

Ansible for Beginners: Create Your First Inventory File

1. What Is an Inventory File?

- An inventory file tells Ansible which servers to manage.
- It lists the IP addresses or hostnames of your target servers.
- You can organize servers into groups (e.g. webservers, dbservers).
- Ansible uses this file when you run a command or playbook.

2. Inventory File Structure

An inventory file can be written in INI format or YAML format. In this page, we'll use the simple INI format.



inventory.ini

3. Create an inventory.ini File

```
# inventory.ini
[webservers]
server1 ansible_host=192.168.1.10
        ansible_user=ubuntu
server2 ansible_host=192.168.1.11
        ansible_user=ubuntu

[dbservers]
db1   ansible_host=192.168.1.20
      ansible_user=ubuntu
```

4. What the Inventory File Means

Part	Explenation
[webservers]	A group name. All servers under this section are web servers.
server1	A name you give to the server.
ansible_host	The server's IP address or hostname.
ansible_user	The SSH username Ansible will use.
[dbservers]	Another group for database servers.
db1	A server in the dbservers group.

5. View Your Inventory

```
$ ansible-inventory -i inventory.ini --graph
@all:
 |--@webservers:
 |   |--server1
 |   |--server2
 |--@dbservers:
 |   |--db1
```

This shows all groups and servers in a tree structure.

6. Why Grouping Servers Matters

- ✓ You can run commands on a specific group (e.g. webservers) instead of all servers.
- ✓ You can keep development, staging, and production servers separate.
- ✓ It reduces the risk of running changes on the wrong servers.
- ✓ It makes playbooks easier to manage.
- ✓ You can apply different configurations to different groups.

7. Example: Run a Command on a Group

```
$ ansible webservers -i inventory.ini -m ping
# This runs the ping module on all servers
# in the webservers group.
```

8. Best Practices

- ✓ Use meaningful group names (web, db, dev, prod).
- ✓ Keep development and production in separate groups.
- ✓ Use hostnames instead of IP addresses when possible.
- ✓ Store your inventory file in your project directory.
- ✓ Use variables (ansible_user, ansible_host) for flexibility.

Ansible for Beginners: Connect to Your Server Using SSH

1. What Is SSH?

- SSH (Secure Shell) is a **secure** way to connect to a remote Linux server.
- It **encrypts** your connection and keeps your information safe.
- DevOps engineers use SSH to **run commands**, **manage files**, and allow tools like Ansible to connect to servers.

2. SSH Authentication Methods

Password Authentication

- Uses a username and password.
- Easy to set up but less secure.
- Not recommended for production.

SSH Key Authentication

- Uses a private key and public key.
- More secure and recommended.
- Commonly used for Ansible and automation.

3. SSH Key Concept



4. Principle of Least Privilege

- Use a **non-root user** for daily tasks (e.g. **ubuntu**, **ec2-user**, centos).
- Only use **root or sudo** when necessary.
- Limits damage if your account is compromised.
- Follow security best practices.

5. Connect to Your Server Using SSH

```
ssh ubuntu@192.168.1.10
# Example using a private key
ssh -i ~/.ssh/id_rsa ubuntu@192.168.1.10
```

Replace with your username and server IP address.

Common SSH Options

`-i <key>`	Use a specific private key
`-p <port>`	Connect to a different port
`-v`	Show detailed connection logs

6. Test Ansible Connectivity With the Ping Module

```
ansible webservers -i inventory.ini -m ansible.builtin.ping
```

Runs the ping module on all servers in the webservers group.

7. Example Output (Successful Connection)

```
server1 | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
```

This means Ansible successfully connected to the server over SSH and executed a module.

8. Why Ansible Ping Is Different

- The Ansible ping module does **NOT** send an **ICMP** network ping (like the ping command).
- It runs a small **Python module** on the remote server over SSH.
- It checks if Ansible can **connect, authenticate, and execute a module.**
- If you see **pong**, it means Ansible can run tasks on that server.

9. Common Connection Problems

- Permission denied (wrong key or username)
- Host unreachable (wrong IP or network issue)
- SSH port blocked (firewall)
- Wrong user (e.g. use ubuntu, ec2-user, or centos)

10. Next Step

- ✓ Once the ping test works, you are ready to run playbooks on this server or group of servers.
- ✓ Verify that all servers in your inventory can connect.
- ✓ Move to creating your first playbook.

Ansible for Beginners: Your First YAML File: Syntax That Ansible Understands

1. What Is YAML?

- YAML (YAML Ain't Markup Language) is a human-readable format used to write Ansible playbooks.
- It uses indentation to define structure.
- It is simple, clean, and easy to read.
- YAML is case-sensitive.
- Ansible playbooks are written in YAML.

2. Basic YAML Syntax

Key-Value Pair	`name: value`
List	`- item1` `- item2`
String	`message: "Hello World"` or `message: Hello World`
Number	`port: 80`
Boolean	`enabled: true` (or false)
Comment	`# This is a comment`

3. Indentation in YAML

- YAML uses spaces, not tabs.
- Indentation shows the relationship between keys and values.
- Use 2 spaces for each level of indentation (recommended).
- Do not use tabs. Tabs can cause errors.
- Keep indentation consistent.

Correct Indentation (2 spaces)

```yaml
- name: Install Nginx
  hosts: webservers
  become: true
  tasks:
    - name: Install package
      apt:
        name: nginx
        state: present
```

Incorrect Indentation (Will Fail)

```yaml
- name: Install Nginx
  hosts: webservers
  become: true
  tasks:
  - name: Install package
      apt:
        name: nginx
        state: present
```

4. YAML Examples in Ansible

Key-Value Pair Example

```yaml
name: Install Nginx
state: present
```

List Example

```yaml
packages:
  - nginx
  - git
  - curl
```

String Example

```yaml
message: "Welcome
 to VERIQTA"
```

Boolean Example

```yaml
enabled: true
state: false
```

5. A Complete YAML Playbook Example

```yaml
---
- name: My First Playbook
  hosts: webservers
  become: true
  tasks:
    - name: Install Nginx
      apt:
        name: nginx
        state: present
    - name: Start and enable Nginx
      service:
        name: nginx
        state: started
        enabled: true
```

Explanation

1. `---` : Start of the YAML file.
2. `name:` Play name.
3. `hosts:` Target servers (from your inventory).
4. `become: true` : Run tasks with sudo.
5. `tasks:` List of tasks to execute.
6. Each task has a name and a module (e.g. apt, service).
7. Indentation shows which lines belong together.

6. Common YAML Mistakes

- Using tabs instead of spaces.
- Wrong indentation.
- Missing colons (:).
- Incorrect nesting of lists and dictionaries.
- Typos in keys or module names.
- Mixing spaces inconsistently.

7. Why YAML Errors Happen

- YAML is strict about indentation.
- Tabs are not allowed.
- Misplaced spaces break the structure.
- Incorrect nesting makes Ansible unable to understand the playbook.
- Even a small indentation mistake can cause a playbook failure.

Ansible for Beginners: Write Your First Ansible Playbook

1. What Is an Ansible Playbook?

- A playbook is a YAML file that tells Ansible what to do, on which servers, and in which order.
- It contains one or more plays.
- Each play has tasks that call Ansible modules.
- Playbooks make your automation repeatable, consistent, and easy to manage.

2. Structure of a Playbook

name	A friendly name for the play.
hosts	The target servers (from inventory).
gather_facts	Collects system information (facts) about the target server.
tasks	A list of actions to perform.
module	The actual work to do (e.g. debug, apt, service, copy).

3. Create Your First Playbook File

first-playbook.yml

```
---
- name: My First Ansible Playbook      ← Play name
  hosts: webservers                    ← Target servers (group)
  gather_facts: true                   ← Collect system facts
  tasks:                               ← List of tasks
    - name: Say hello                  ← Task name
      ansible.builtin.debug:           ← Ansible module
        msg: "Hello from Ansible!"     ← Message to display
```

4. Explanation of Each Part

Field	Why It Exists
name	Helps you identify the play when you run it.
hosts	Specifies which servers to run the play on (uses inventory).
gather_facts	Collects useful information about the server (OS, IP, etc).
tasks	A list of actions to perform on the target servers.
ansible.builtin.debug	A built-in module used to show a message. Good for testing and learning.

5. Run Your Playbook

```
$ ansible-playbook -i inventory.ini first-playbook.yml
```

6. Example Output

```
PLAY [My First Ansible Playbook] ********************

TASK [Gathering Facts] ******************************
ok: [server1]
ok: [server2]

TASK [Say hello] ************************************
ok: [server1] => {
    "msg": "Hello from Ansible!"
}
ok: [server2] => {
    "msg": "Hello from Ansible!"
}

PLAY RECAP ******************************************
server1      : ok=2   changed=0   unreachable=0   failed=0
server2      : ok=2   changed=0   unreachable=0   failed=0
```

7. What Happens When You Run It?

1. Ansible reads the inventory.ini file.
2. It finds the servers in the webservers group.
3. It connects to each server over SSH.
4. It gathers system facts.
5. It runs the debug task and displays the message.
6. It shows a recap with the result for each server.

8. Important Notes

- Make sure your inventory file has the correct server IP addresses and usernames.
- The servers must be reachable over SSH.
- You can set gather_facts: false if you don't need system facts.
- This is a simple playbook to test your setup. You can add more tasks later (install packages, start services, copy files, etc).

9. Key Takeaways

- A playbook is a YAML file.
- It has a name, hosts, gather_facts, and tasks.
- You run it with ansible-playbook.
- The debug module helps you test your setup.
- The play recap shows the result for each server.

Ansible for Beginners: Understand Tasks, Modules, and Playbook Results

1. What Are Tasks and Modules?

- A **task** is a single action in a playbook.
- A **module** is the actual code that performs the action on the target server.
- Each task calls a module (e.g. **apt**, **service**, **copy**, **debug**).
- Ansible modules are **idempotent**, which means running them multiple times will not make unnecessary changes.

2. How a Task Calls a Module



1. You define a task in your playbook.
2. The task calls an Ansible module.
3. Ansible connects to the target server over SSH.
4. The module performs the action and returns a result (ok, changed, failed, or skipped).

3. Common Modules for Beginners

ansible.builtin.apt

Used to manage packages on Ubuntu/Debian.

```yaml
- name: Install Nginx
  ansible.builtin.apt:
    name: nginx
    state: present
    update_cache: true
```

ansible.builtin.service

Used to manage services (e.g. start, stop, enable).

```yaml
- name: Start and enable Nginx
  ansible.builtin.service:
    name: nginx
    state: started
    enabled: true
```

4. Understanding Task Results

Result	Meaning	Example
ok	Task ran successfully but made no changes.	ok: [server1]
changed	Task ran successfully and made a change on the server.	changed: [server1]
failed	Task failed. Check the error message.	failed: [server1]
skipped	Task was skipped (e.g. due to a condition).	skipped: [server1]

5. Example Playbook With Multiple Tasks

```yaml
---
- name: Install and start Nginx
  hosts: webservers
  become: true
  tasks:
    - name: Install Nginx package
      ansible.builtin.apt:
        name: nginx
        state: present
        update_cache: true

    - name: Start and enable Nginx service
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true
```

6. Run the Playbook

```
$ ansible-playbook -i inventory.ini first-playbook.yml
```

7. Example Playbook Output

```
PLAY [Install and start Nginx] ***************************

TASK [Gathering Facts] ***********************************
ok: [server1]

TASK [Install Nginx package] *****************************
changed: [server1]

TASK [Start and enable Nginx service] ********************
ok: [server1]

PLAY RECAP ***********************************************
server1    : ok=3  changed=1  unreachable=0  failed=0  skipped=0
```

8. How to Read the Play Recap

Field	Meaning	Example
ok	Number of successful tasks.	ok=3
changed	Number of tasks that made changes.	changed=1
unreachable	Number of unreachable servers.	unreachable=0
failed	Number of failed tasks.	failed=0
skipped	Number of skipped tasks.	skipped=0

9. Key Takeaways

- A playbook is a list of tasks.
- Each task calls a specific module.
- Modules perform the actual work on the target server.
- **ok**, **changed**, **failed**, and **skipped** tell you the result.
- Use **ansible.builtin.apt** to manage packages.
- Use **ansible.builtin.service** to manage services.
- Check the play recap to see if the server was changed.
- If a task shows changed, it means something was modified.

Ansible for Beginners: Install Nginx Using Your First Real Automation Task

1. What We Will Do in This Page

- Extend our playbook to install Nginx on an Ubuntu server.
- Use ansible.builtin.apt to install the package.
- Use become: true to run the task with administrative privileges.
- Verify the installation and check the service status.
- Explain why elevated permissions are required.

2. Why We Need Elevated Permissions



- Installing software, changing system files, and managing services require root privileges.
- Regular users cannot install packages or start system services.
- We use become: true so Ansible can run the task with administrative privileges (usually with sudo).

3. Playbook: Install Nginx

install-nginx.yml

```yaml
---
- name: Install Nginx on Ubuntu
  hosts: webservers
  become: true
  tasks:
    - name: Update package cache
      ansible.builtin.apt:
        update_cache: true

    - name: Install Nginx package
      ansible.builtin.apt:
        name: nginx
        state: present
```

4. Explanation of Key Fields

Field	Purpose
name	A friendly name for the play or task.
hosts	The target servers (from your inventory).
become	Allows the task to run with elevated privileges.
tasks	A list of tasks to execute.
ansible.builtin.apt	Ansible module to manage packages on Ubuntu/Debian.
update_cache	Updates the package cache before installation.
name: nginx	Specifies the package to install.
state: present	Ensures the package is installed.

5. Run the Playbook

```
$ ansible-playbook -i inventory.ini install-nginx.yml
```

6. Example Playbook Output

```
PLAY [Install Nginx on Ubuntu] ************************
TASK [Update package cache] ***************************
ok: [server1]
ok: [server2]
TASK [Install Nginx package] **************************
changed: [server1]
changed: [server2]
PLAY RECAP ********************************************
server1      : ok=2  changed=1  unreachable=0  failed=0
server2      : ok=2  changed=1  unreachable=0  failed=0
```

7. Verify the Installation

Check Nginx package

```
$ ansible webservers -i inventory.ini -m shell -a
 "dpkg -l | grep nginx"
```

Check Nginx service status

```
$ ansible webservers -i inventory.ini -m shell -a
 "systemctl status nginx --no-pager"
```

8. Expected Verification Output (Example)

```
server1 | CHANGED | rc=0 >>
ii  nginx   1.24.0-1ubuntu1  amd64  high performance web server

server1 | CHANGED | rc=0 >>
● nginx.service - A high performance web server
     Active: active (running) since ...
   Main PID: 1234 (nginx)
```

9. Important Notes

- Package installation and service management require elevated privileges (root or sudo).
- become: true uses sudo to run the task as an administrator.
- Without elevated permissions, the task will fail with a permission error.
- The apt module is idempotent, meaning it will not reinstall Nginx if it is already installed.

10. Key Takeaways

- We installed Nginx using ansible.builtin.apt.
- We used become: true for administrative privileges.
- We verified the installation and service status.
- Elevated permissions are required for system-level changes.
- The task is idempotent and safe to run multiple times.

Ansible for Beginners: Manage Linux Services and Verify Nginx

1. What We Will Do in This Page

- Use ansible.builtin.systemd_service to manage the Nginx service.
- Ensure Nginx is running and enabled at boot.
- Explain state: started and enabled: true.
- Verify the service status using systemctl and check the website using curl.
- Understand the difference between a running service and a successfully responding website.

2. Understanding Service States

Parameter	Meaning
name	The name of the service (nginx).
state: started	Ensures the service is running. Starts it if it is not running.
state: stopped	Stops the service.
state: restarted	Restarts the service.
enabled: true	Starts the service now and enables it to start automatically at boot.
enabled: false	Disables the service from starting at boot.

3. Playbook: Manage and Start Nginx Service

nginx-service.yml

```
---
- name: Ensure Nginx is running and enabled
  hosts: webservers
  become: true
  tasks:
    - name: Start and enable Nginx service
      ansible.builtin.systemd_service:
        name: nginx
        state: started
        enabled: true
```

4. How It Works

1. Ansible connects to the server over SSH and runs the systemd_service module.
2. It checks if Nginx is installed (from the previous page).
3. If Nginx is not running, it starts the service (state: started).
4. It enables the service to start automatically on server reboot (enabled: true).
5. The module is idempotent, so running it again will not make unnecessary changes if the service is already running and enabled.

5. Run the Playbook

```
$ ansible-playbook -i inventory.ini nginx-service.yml
```

6. Example Playbook Output

```
PLAY [Ensure Nginx is running and enabled] ************
TASK [Start and enable Nginx service] *****************
changed: [server1]
ok: [server2]
PLAY RECAP ********************************************
server1    : ok=1   changed=1   unreachable=0   failed=0
server2    : ok=1   changed=0   unreachable=0   failed=0
```

7. Verify the Service on the Server

1. Check if Nginx service is running

```
$ ansible webservers -i inventory.ini -m shell -a
"systemctl is-active nginx"
```

2. Check if Nginx is enabled at boot

```
$ ansible webservers -i inventory.ini -m shell -a
"systemctl is-enabled nginx"
```

8. Verify the Website Using curl

```
$ ansible webservers -i inventory.ini -m shell -a
"curl -I http://localhost"
HTTP/1.1 200 OK
Server: nginx/1.24.0 (Ubuntu)
Content-Type: text/html
```

A 200 OK response means the website is responding correctly.

9. Running Service vs. Working Website

Running Service

- Nginx process is active (running).
- Checked with: systemctl is-active nginx
- Means the service is running on the server.
- Does not confirm that the website is accessible from a browser.

Successfully Responding Website

- Nginx is running and serving web content.
- Checked with: curl or a web browser.
- You receive an HTTP 200 OK response.
- Confirms that Nginx is working and returning a web page.

10. Key Takeaways

- Use ansible.builtin.systemd_service to manage services.
- state: started ensures the service is running.
- enabled: true enables auto start at boot.
- Verify with systemctl and curl.
- A running service is not the same as a working website.
- The task is idempotent and safe to run multiple times.

Ansible for Beginners: Variables: Stop Hardcoding Your Configuration

1. What Are Variables?

- Variables let you **store values** and reuse them in your playbooks.
- Instead of hardcoding values like package names, paths, or settings, you use variables.
- Variables make your automation **reusable, flexible, and easy to manage**.
- Ansible uses **Jinja2 syntax {{ variable_name }}** to reference variables.

2. Common Variables in a Web Server Setup

Variable	Example Value	Purpose
web_package	nginx	The package to install.
web_service	nginx	The service name.
web_dir	/var/www/html	The web directory path.
app_port	80	The application port.
app_name	My Website	A custom application name or message.

3. Example Playbook Using Variables

variables-example.yml

```yaml
---
- name: Install and configure Nginx using variables
  hosts: webservers
  become: true
  vars:
    web_package: nginx
    web_service: nginx
    web_dir: /var/www/html
    app_port: 80
    app_name: "Welcome to VERIQTA"
  tasks:
    - name: Install Nginx package
      ansible.builtin.apt:
        name: "{{ web_package }}"
        state: present
        update_cache: true
```

4. Using Variables in Tasks

```yaml
- name: Ensure Nginx is running and enabled
  ansible.builtin.systemd_service:
    name: "{{ web_service }}"
    state: started
    enabled: true

- name: Create a custom index.html
  ansible.builtin.copy:
    dest: "{{ web_dir }}/index.html"
    content: |
      <h1>{{ app_name }}</h1>
      <p>Running on port {{ app_port }}</p>
```

5. Run the Playbook

```
$ ansible-playbook -i inventory.ini variables-example.yml
```

6. Expected Result

```
TASK [Install Nginx package] ************************
ok: [server1]
TASK [Ensure Nginx is running and enabled] ***********
ok: [server1]
TASK [Create a custom index.html] ********************
changed: [server1]
```

7. Verify the Result

Check Nginx service status

```
$ ansible webservers -i inventory.ini -m shell -a
 "systemctl is-active nginx"
  server1 | CHANGED | rc=0 >>
     active
```

Check the web page

```
$ ansible webservers -i inventory.ini -m shell -a
 "curl -I http://localhost"
  server1 | CHANGED | rc=0 >>
     HTTP/1.1 200 OK
```

8. Why Variables Are Important

- You can reuse the same playbook for different servers.
- You can easily change values without editing tasks.
- Makes your automation consistent and easier to maintain.
- You can use different variables for development, testing, and production.
- Supports environment-specific configurations.
- Reduces the risk of mistakes.

9. Do Not Store Passwords in Plain Text

- Never store real passwords, API keys, or sensitive data directly in playbooks.
- Use Ansible Vault to encrypt sensitive variables.
- Alternatively, use environment variables or a secure secrets manager.
- Keeping secrets in plain text is a security risk and can expose your servers.

10. Key Takeaways

- Variables store values and make playbooks reusable.
- Use Jinja2 syntax {{ variable_name }}.
- You can define variables under vars:.
- Variables help you manage different environments.
- Never store passwords or sensitive data in plain text.
- Use Ansible Vault or a secure method for secrets.

Ansible for Beginners: Deploy Website Files Using Copy and Templates

1. What We Will Do in This Page

- Use ansible.builtin.copy and ansible.builtin.template to deploy website files.
- Deploy a simple HTML homepage to /var/www/html/index.html.
- Understand file ownership, permissions, and destination paths.
- Learn how Jinja2 templates insert variables into website files.

2. Copy vs Template: What's the Difference?

ansible.builtin.copy

- Copies a file as it is.
- No variable processing.
- Use for static files (e.g. images, HTML files).
- Simple and fast.

ansible.builtin.template

- Uses Jinja2 to insert variables.
- Good for configuration files and dynamic content.
- Use {{ variable_name }} in the file.
- Creates a customized file on the target server.

3. Example: Using ansible.builtin.copy

copy-website.yml

```yaml
---
- name: Deploy a simple HTML homepage
  hosts: webservers
  become: true
  tasks:
    - name: Copy index.html to web directory
      ansible.builtin.copy:
        src: files/index.html
        dest: /var/www/html/index.html
        owner: www-data
        group: www-data
        mode: '0644'
```

4. Example: Using ansible.builtin.template

template-website.yml

```yaml
---
- name: Deploy homepage using a template
  hosts: webservers
  become: true
  vars:
    site_title: "Welcome to VERIQTA"
    site_message: "Automate your infrastructure"
  tasks:
    - name: Deploy index.html from template
      ansible.builtin.template:
        src: templates/index.html.j2
        dest: /var/www/html/index.html
        owner: www-data
        group: www-data
        mode: '0644'
```

5. Example Template File (templates/index.html.j2)

index.html.j2

```
<!DOCTYPE html>
<html>
  <head>
    <title>{{ site_title }}</title>
  </head>
  <body>
    <h1>{{ site_title }}</h1>
    <p>{{ site_message }}</p>
  </body>
</html>
```

6. Variables in the Deployed File (Result)

Deployed /var/www/html/index.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>Welcome to VERIQTA</title>
  </head>
  <body>
    <h1>Welcome to VERIQTA</h1>
    <p>Automate your infrastructure</p>
  </body>
</html>
```

7. Important Settings Explained

Parameter	Meaning
src	Path to the source file on your control node.
dest	Destination path on the target server.
owner	File owner (e.g. www-data).
group	File group (e.g. www-data).
mode	File permissions (e.g. '0644').
{{ variable }}	Jinja2 variable that will be replaced with a value.
templates/	Folder that stores Jinja2 template files (.j2).
files/	Folder that stores static files for copy module.

8. Verify the Deployment

```
# Check if the file exists
ansible webservers -i inventory.ini -m shell -a
"ls -l /var/www/html/index.html"

# Test the website
ansible webservers -i inventory.ini -m shell -a
"curl -I http://localhost"
```

9. Why Use Templates and Variables?

- You can use the same playbook for different environments.
- Variables make automation reusable and flexible.
- Templates allow you to generate dynamic configuration files.
- You can change values without editing the template file.

10. Security Note

- Do not store passwords, API keys, or sensitive data in plain text.
- Use Ansible Vault to encrypt sensitive information.
- Keep your code and variables secure in version control.

Ansible for Beginners: Handlers: Restart Services Only When Necessary

1. What Are Handlers?

- Handlers are special tasks that run **only** when **notified** by another task.
- They are usually used to **restart or reload** services after a configuration file changes.
- Handlers help you **avoid unnecessary service restarts**.
- They run **at the end of the play**, only if the task that notified them reports a change (changed).

2. Tasks vs Handlers

Feature	Ordinary Tasks	Handlers
Purpose	Perform general work (e.g. copy files, install packages).	Used for actions like restarting or reloading services.
When they run	Run every time the playbook runs.	Run only when notified and only if there was a change.
Triggered by	Run directly in order.	Triggered using notify from another task.
Common use	Install software, copy files, create users, etc.	Restart or reload services (e.g. Nginx).

3. Example Playbook with a Handler

nginx-handler.yml

```yaml
---
- name: Deploy Nginx configuration with handler
  hosts: webservers
  become: true
  tasks:
    - name: Deploy Nginx configuration file
      ansible.builtin.template:
        src: templates/nginx.conf.j2
        dest: /etc/nginx/nginx.conf
        owner: root
        group: root
        mode: '0644'
        notify: Restart Nginx

  handlers:
    - name: Restart Nginx
      ansible.builtin.systemd_service:
        name: nginx
        state: reloaded
```

4. How It Works

1. The **template** task copies the Nginx configuration file to the server.
2. If the file is different, the task reports **changed**.
3. The task uses **notify: Restart Nginx** to tell Ansible to run the handler.
4. The handler runs **only once** at the end of the play, even if multiple tasks notify it.
5. If the file has **not changed**, the handler **will not run**, so Nginx is not restarted.

5. Example Template File (templates/nginx.conf.j2)

nginx.conf.j2

```
server {
    listen 80;
    server_name {{ server_name }};

    root {{ web_root }};
    index index.html;
    location / {
        try_files $uri $uri/ =404;
    }
}
```

6. What Happens When You Run It?

```
$ ansible-playbook -i inventory.ini nginx-handler.yml

TASK [Deploy Nginx configuration file] ***************
changed: [server1]

RUNNING HANDLER [Restart Nginx] **********************
ok: [server1]

PLAY RECAP *******************************************
server1      : ok=2   changed=1   unreachable=0   failed=0
server2      : ok=2   changed=0   unreachable=0   failed=0
```

7. Why Handlers Are Important

- Avoid unnecessary service restarts, which can cause brief downtime or interrupt active connections.
- Improve stability and reliability.
- Make automation more efficient and predictable.
- Ensure services are restarted or reloaded only when configuration files actually change.
- Support controlled and safe changes in production systems.

8. Avoidable Disruption Example

Without Handlers

- Service restarts every time you run the playbook.
- Can cause unnecessary downtime and interrupt users.
- Not efficient and can lead to instability.

With Handlers

- Service restarts only when the configuration file changes.
- No unnecessary downtime.
- More stable and production-ready.
- Best practice for managing services like Nginx, Apache, etc.

9. Key Takeaways

- Handlers run only when notified and only if there is a change.
- Use **notify** to trigger a handler from a task.
- Use handlers for restarting or reloading services.
- This prevents unnecessary restarts and reduces disruption.
- Handlers run at the **end of the play** and only once.
- Always use handlers for service management in real environments.

Ansible for Beginners: Idempotency: Why Running a Playbook Twice Matters

1. What Is Idempotency?

- Idempotency means your playbook can run **multiple times** and still reach the **same desired state** without making unnecessary changes.
- If the system is already in the correct state, Ansible will **not make changes again**.
- This prevents **configuration drift** and makes automation **safe** and **repeatable**.
- Ansible achieves this by using **declarative state** (for example, **state: present, state: started**).

2. Why Idempotency Matters

1. Safe **to run repeatedly** without breaking your servers.
2. Ensures **consistent server configuration** across environments.
3. Critical for **CI/CD pipelines** where playbooks run automatically.
4. Helps with **server recovery** (you can rebuild or reconfigure anytime).
5. Reduces unnecessary changes, server downtime, and operational risk.



3. Example Playbook (Idempotent)

idempotent-nginx.yml

```yaml
---
- name: Install and ensure Nginx is running
  hosts: webservers
  become: true
  tasks:
    - name: Install Nginx package
      ansible.builtin.apt:
        name: nginx
        state: present
        update_cache: true

    - name: Ensure Nginx service is running
      ansible.builtin.systemd_service:
        name: nginx
        state: started
        enabled: true
```

4. First-Run Output (Changes Made)

```
$ ansible-playbook -i inventory.ini idempotent-nginx.yml

TASK [Install Nginx package] ***************************
changed: [server1]
changed: [server2]

TASK [Ensure Nginx service is running] *****************
changed: [server1]
changed: [server2]

PLAY RECAP *********************************************
server1    : ok=2   changed=2   unreachable=0   failed=0
server2    : ok=2   changed=2   unreachable=0   failed=0
```

5. Second-Run Output (No Unnecessary Changes)

```
$ ansible-playbook -i inventory.ini idempotent-nginx.yml

TASK [Install Nginx package] ***************************
ok: [server1]
ok: [server2]

TASK [Ensure Nginx service is running] *****************
ok: [server1]
ok: [server2]

PLAY RECAP *********************************************
server1    : ok=2   changed=0   unreachable=0   failed=0
server2    : ok=2   changed=0   unreachable=0   failed=0
```

6. Understanding the Results

Result	Meaning	Example
ok	Task ran successfully but no changes were needed.	ok: [server1]
changed	Task made a change on the server.	changed: [server1]
failed	Task failed. Check the error message.	failed: [server1]
skipped	Task was skipped (e.g. due to a condition).	skipped: [server1]

7. Why changed=0 on the Second Run?

- Ansible checks the current state before making changes.
- If the package is already installed, it does not install it again.
- If the service is already running, it does not restart it again.
- This is idempotency: the playbook reaches the desired state and leaves it unchanged.

8. Real-World Benefits

- Safe to run in CI/CD pipelines (e.g. on every deployment).
- Useful for server recovery and disaster scenarios.
- Keeps environments consistent (dev, staging, production).
- Prevents unnecessary service restarts and downtime.
- Makes automation predictable and reliable.

9. Key Takeaways

- A well-designed Ansible playbook can run multiple times safely.
- Idempotency ensures changed=0 when the desired state is already met.
- Use declarative state (e.g. present, started) in your tasks.
- Repeatability is essential for CI/CD, recovery, and consistent infrastructure.

Ansible for Beginners: Conditions, Loops, and Facts

1. What We Will Do in This Page

- Use **when** to run tasks only when conditions are met.
- Use **loop** to run the same task for multiple items (e.g. install multiple packages).
- Use **ansible_facts** to get system information (e.g. OS, version, IP address).
- See how a playbook can make decisions based on the operating system.
- Understand why running every task on every host can cause failures.

2. Key Concepts

Concept	Explanation
when	Runs a task only if a condition is true. Helps you target the right systems.
loop	Repeats a task for multiple items. Useful for installing multiple packages or creating multiple users.
ansible_facts	System information collected by Ansible (e.g. operating system, IP address, memory, etc.).
register	Stores the result of a task in a variable so you can use it later (optional, but useful).

3. Example: Using when (Run Task Based on OS)

when-example.yml

```yaml
---
- name: Install Nginx only on Ubuntu
  hosts: webservers
  become: true
  tasks:
    - name: Install Nginx on Ubuntu
      ansible.builtin.apt:
        name: nginx
        state: present
      when: ansible_facts['os_family'] == "Debian"
```

4. Example: Using loop (Install Multiple Packages)

loop-example.yml

```yaml
---
- name: Install multiple packages
  hosts: webservers
  become: true
  tasks:
    - name: Install essential packages
      ansible.builtin.apt:
        name: "{{ item }}"
        state: present
      loop:
        - nginx
        - git
        - htop
```

5. Example: Using ansible_facts (Show System Info)

facts-example.yml

```yaml
---
- name: Show system information
  hosts: webservers
  tasks:
    - name: Display OS and IP address
      ansible.builtin.debug:
        msg: "OS: {{ ansible_facts['os_family'] }}
        Version: {{ ansible_facts['distribution_version'] }}
        IP: {{ ansible_facts['default_ipv4']['address'] }}"
```

6. Sample Output (Facts)

ansible-playbook facts-example.yml

```
TASK [Display OS and IP address] ********************
ok: [server1] => {
    "msg": "OS: Debian\nVersion: 12\nIP: 192.168.1.10"
}
ok: [server2] => {
    "msg": "OS: Ubuntu\nVersion: 22.04\nIP: 192.168.1.11"
}
```

7. Why Conditions and Loops Are Important

- Different servers may run different operating systems (Ubuntu, RHEL, etc.).
- Running the wrong package manager can cause task failures.
- Conditions (when) ensure tasks run only where they are valid.
- Loops help you manage multiple items without repeating code.
- Using ansible_facts allows your playbook to make intelligent decisions based on real system information.

8. What Happens If You Run Every Task on Every Host?

- You may try to use the wrong package manager (e.g. apt on RHEL).
- Tasks can fail and stop the playbook.
- You may install unnecessary packages on servers that don't need them.
- It can cause configuration drift and make troubleshooting harder.
- Your automation becomes less reliable and harder to maintain.

9. Key Takeaways

- Use **when** to run tasks only when conditions are met.
- Use **loop** to repeat tasks for multiple items.
- Use **ansible_facts** to get real system information.
- Make your playbooks smart, flexible, and environment-aware.
- Avoid running every task on every host to prevent failures.

Ansible for Beginners: Troubleshoot Failed Ansible Playbooks

1. What We Will Do in This Page

- Learn how to troubleshoot common Ansible errors.
- Investigate UNREACHABLE, Permission denied, YAML syntax errors, missing packages, and privilege escalation failures.
- Use --syntax-check, -vv, and targeted SSH tests to find the real cause.
- Learn to inspect the failing task and error message before changing anything.

2. Common Ansible Errors and What They Mean

Error	What It Means	Common Cause
UNREACHABLE	Ansible cannot connect to the host.	Wrong IP, SSH not running, firewall, wrong user or key.
Permission denied	Authentication failed or insufficient privileges.	Wrong SSH key, wrong user, or missing become: true.
YAML syntax error	Playbook file is not valid YAML.	Incorrect indentation, missing : or -.
Package not found	The package name does not exist.	Wrong package name or wrong OS.
Privilege escalation failure	Cannot become root using sudo.	Missing sudo rights, wrong become method, or password required.

3. Example: UNREACHABLE Error

```
$ ansible-playbook -i inventory.ini site.yml
TASK [Gathering Facts] **********************
fatal: [server1]: UNREACHABLE! => {
  "msg": "Failed to connect to the host via ssh:
} Connection timed out"
```

How to fix:

- Check the IP address and hostname.
- Ensure SSH is running: systemctl status ssh
- Test connection manually: ssh user@server1
- Check firewall and security group rules.

4. Example: Permission Denied

```
$ ansible-playbook -i inventory.ini site.yml
TASK [Gathering Facts] ********************
fatal: [server1]: FAILED! => {
  "msg": "Permission denied (publickey)."
}
```

How to fix:

- Use the correct SSH user and key.
- Check key permissions (chmod 600 key.pem).
- Verify the user has access to the server.
- If using become, ensure the user has sudo rights.

5. Example: YAML Syntax Error

```
$ ansible-playbook site.yml
ERROR! while parsing a block mapping
expected <block end>, but found '-'
 in "site.yml", line 6, column 3
```

How to fix:

- Check indentation (use spaces, not tabs).
- Ensure every key has a colon (:).
- Validate the playbook:

```
$ ansible-playbook --syntax-check site.yml
```

6. Example: Package Not Found

```
TASK [Install package] **********************
fatal: [server1]: FAILED! => {
  "msg": "No package matching 'nginxg' is
} available"
```

How to fix:

- Check the correct package name (e.g. nginx).
- Ensure you are using the right OS and package manager.
- Update package cache: apt update (or yum/dnf).
- Verify internet access on the server.

7. Example: Privilege Escalation Failure

```
TASK [Install package] ******************
fatal: [server1]: FAILED! => {
  "msg": "Missing sudo password"
}
```

How to fix:

- Use become: true in your playbook.
- Ensure the user has sudo privileges.
- Configure become_method: sudo (default).
- Use an SSH user that can run sudo without a password or provide a secure method (e.g. Ansible Vault for passwords).

8. Useful Commands for Troubleshooting

```
# Check playbook syntax
$ ansible-playbook --syntax-check site.yml
# Run with verbose output (more details)
$ ansible-playbook -vv site.yml
# Run with even more details
$ ansible-playbook -vvv site.yml
# Test SSH connection manually
$ ssh user@server1
# Test Ansible connection to a specific host
$ ansible server1 -m ping -i inventory.ini
# Run only a specific task (by tag)
$ ansible-playbook site.yml --tags "nginx"
```

9. Troubleshooting Process (Always Follow This Order)

1. Read the full error message (it tells you what failed).
2. Identify the failing task.
3. Understand the reason (connection, permissions, syntax, package, etc.).
4. Check your inventory and variables.
5. Test the issue manually (SSH, command, etc.).
6. Fix one thing at a time and run the playbook again.

10. Key Takeaways

- Always read the full error message before making changes.
- Use --syntax-check to catch YAML errors early.
- Use -vv for detailed output when troubleshooting.
- Test SSH access manually to confirm connectivity.
- Common failures are related to connection, permissions, package names, or incorrect configuration.
- Fixed errors make your automation more reliable and predictable.

Check Mode, Diff Mode, and Safer Changes

1. What We Will Do in This Page

- Use **--check** to preview what would change without making changes.
- Use **--diff** to inspect file differences before applying changes.
- Use **--limit** to restrict execution to selected hosts.
- Understand that check mode is **not a complete simulation** and must not replace real testing and verification.

2. The Three Options

Option	What It Does	Example
--check	Runs tasks in check mode (does not make changes).	ansible-playbook site.yml --check
--diff	Shows file differences before changes are applied.	ansible-playbook site.yml --diff
--limit	Runs the playbook on specific hosts only.	ansible-playbook site.yml --limit web-01

Check mode is not a complete simulation.
It cannot catch everything. Always run real tests and verify the results.

3. Check Mode Example

```
ansible-playbook site.yml --check

PLAY [webservers] **************************

TASK [Install nginx] ***********************
changed: [web-01]

TASK [Ensure nginx is running] ***************
ok: [web-01]

PLAY RECAP *********************************
web-01       : ok=2    changed=1   unreachable=0
               failed=0  skipped=0   rescued=0
```

Shows what would change, but does not apply the changes.

4. Diff Mode Example

```
ansible-playbook site.yml --diff

TASK [Update index.html] *******************
---  before
+++  after
@@  -1,3 +1,3  @@
-    -<title>Old Site</title>
-    +<title>New Site</title>

changed: [web-01]

PLAY RECAP *********************************
web-01         : ok=1    changed=1
                 unreachable=0   failed=0
                 skipped=0   rescued=0
```

Shows the exact file differences before changes are applied.

5. Limit Example

```
ansible-playbook site.yml --limit web-01

PLAY [webservers] **************************

TASK [Install nginx] ***********************
changed: [web-01]

PLAY RECAP *********************************
web-01       : ok=2    changed=1
               unreachable=0   failed=0
               skipped=0   rescued=0
```

Runs the playbook only on selected hosts.

6. What Each Option Helps You Do

--check
- Preview changes before they happen.
- Find issues early.
- Save time and reduce risk.

--diff
- See exactly what will be changed.
- Useful for reviewing file edits.
- Helps with audits and change control.

--limit
- Target specific hosts or groups.
- Fix issues without affecting all servers.
- Useful during troubleshooting and maintenance.

7. Why It Matters

- **Safer changes** – preview and review before applying.
- **Less risk** – avoid unwanted changes on the wrong hosts.
- **Better visibility** – see what will change and why.
- **Faster troubleshooting** – focus on specific hosts or issues.
- **Higher reliability** – combine with real testing and verification.
- **Supports CI/CD** – run checks and diffs in your pipeline before real deployment.

8. Key Takeaways

- --check shows what would change, not what actually changes.
- --diff shows the exact file differences.
- --limit restricts execution to selected hosts.
- Check mode is not a complete simulation. Always test and verify in real environments.

9. Quick Commands

```
ansible-playbook site.yml --check          # Preview changes
ansible-playbook site.yml --diff           # Show file differences
ansible-playbook site.yml --limit web-01   # Run on specific host
ansible --version                          # Check Ansible version
ansible -i inventory all -m ping           # Quick SSH/connection test
```

Ansible for Beginners: Protect Credentials and Organize Your Project

1. Why Protect Credentials?

- Credentials (SSH keys, passwords, API keys) give access to your servers.
- If exposed, attackers can take control of your systems.
- Never store secrets in plain text files.
- Never commit secrets to Git.
- Use proper methods like SSH key protection, no_log, and Ansible Vault.

2. Protect Your SSH Keys

- Use SSH key-based authentication instead of passwords.
- Keep your private key secure (do not share it).
- Set proper permissions: chmod 600 ~/.ssh/id_rsa
- Use a passphrase for extra security.
- Do not commit private keys to Git.

```
# Generate a new SSH key (if needed)
ssh-keygen -t ed25519 -C "your_email@example.com"
# Set correct permissions
chmod 600 ~/.ssh/id_rsa
# Test connection to server
ssh -i ~/.ssh/id_rsa user@server_ip
```



Keep your private key secure!

3. Use no_log to Hide Sensitive Information

- Use no_log: true to prevent sensitive data from appearing in the playbook output.
- Useful when dealing with passwords, tokens, or secret variables.

```
- name: Create a user with a secret password
  ansible.builtin.user:
    name: devuser
    password: "{{ user_password }}"
    state: present
  no_log: true
```

4. Use Ansible Vault to Store Secrets

- Ansible Vault encrypts sensitive files so they can be stored safely in your project.
- Only users with the vault password can view or edit the content.

```
# Create an encrypted file
ansible-vault create group_vars/webservers/vault.yml
# Edit an encrypted file
ansible-vault edit group_vars/webservers/vault.yml
# Use the vault file in a playbook
ansible-playbook site.yml --ask-vault-pass
```



Ansible Vault keeps your secrets encrypted and safe.

5. Simple Project Structure

```
ansible-project/
├── inventory.ini      # Target servers
├── playbook.yml       # Main playbook
├── group_vars/        # Variables (including vault files)
│   ├── all.yml        # Common variables
│   └── webservers/
│       └── vault.yml  # Encrypted secrets (DO NOT edit directly)
├── templates/
│   └── index.html.j2  # Jinja2 template for website files
└── files/             # Static files (optional)
```

6. Which Files Can Be Committed to Git?

File / Folder	Commit to Git?	Notes
inventory.ini	Yes	Safe – contains host information (no secrets).
playbook.yml	Yes	Safe – contains tasks.
group_vars/all.yml	Yes	Safe – non-sensitive variables.
group_vars/webservers/vault.yml	No	Encrypted file – can be committed but keep it encrypted. Never store unencrypted secrets.
templates/	Yes	Safe – contains templates.
files/	Yes	Safe – contains static files.
.ssh/id_rsa	No	Private key – never commit.
Passwords or API keys in plain text	No	Never commit secrets to Git.

7. Key Security Best Practices

- Use SSH keys instead of passwords.
- Keep private keys secure and use a passphrase.
- Use no_log for sensitive data.
- Use Ansible Vault to encrypt secret files.
- Never commit secrets, private keys, or unencrypted vault files to Git.
- Restrict file permissions (chmod 600).
- Review your repository before pushing to Git.

8. Example .gitignore (Protect Sensitive Files)

```
# Python cache files
__pycache__/
*.pyc
# Ansible vault password file (if used)
.vault_pass.txt
# SSH private keys
.ssh/id_rsa
.ssh/id_ed25519
# Any secret files
secrets/
*.vault.yml
```



Always verify your .gitignore to make sure sensitive files are not committed to Git.

Ansible for Beginners: Final Project: Deploy a Website With One Playbook

1. Project Overview

- Bring everything together into one complete deployment workflow.
- Connect to the target server.
- Install Nginx.
- Deploy an HTML page.
- Ensure the service is running and enabled.
- Verify the website responds over HTTP.
- Use the skills you have learned in this series.

2. Architecture Diagram



3. Complete Playbook (site.yml)

```yaml
---
- name: Deploy a website with Nginx
  hosts: webservers
  become: true
  vars:
    web_package: nginx
    web_dir: /var/www/html
    html_file: index.html
    page_content: |
      <html>
        <head><title>Welcome to VERIQTA</title></head>
        <body>
          <h1>Website Deployed with Ansible</h1>
          <p>This is a simple Nginx deployment.</p>
        </body>
      </html>
  tasks:
    - name: Install Nginx
      ansible.builtin.apt:
        name: "{{ web_package }}"
        state: present
    - name: Deploy HTML page
      ansible.builtin.copy:
        content: "{{ page_content }}"
        dest: "{{ web_dir }}/{{ html_file }}"
        owner: root
        group: root
        mode: '0644'
    - name: Ensure Nginx is running and enabled
      ansible.builtin.systemd_service:
        name: nginx
        state: started
        enabled: true
```

4. Run the Playbook

```bash
# Run the playbook
ansible-playbook -i inventory.ini site.yml

# Preview changes first (optional)
ansible-playbook -i inventory.ini site.yml --check --diff

# Run on a specific host (optional)
ansible-playbook -i inventory.ini site.yml --limit web-01
```

5. Verify the Deployment

```bash
# Check Nginx service status
systemctl is-active nginx  # should return: active

# Test the website response
curl http://localhost

# Or test from your local machine
curl http://server_ip
# You should see: Welcome to VERIQTA
```

6. Expected Play Recap (Successful Run)

```
PLAY RECAP *********************************************
web-01          : ok=3    changed=3    unreachable=0
                  failed=0   skipped=0   rescued=0   ignored=0
```

The exact numbers may vary. On a second run you should see changed=0 if nothing needs to be updated.

7. What This Project Achieves

- Installs Nginx on the target server.
- Deploys a custom HTML page.
- Sets correct file ownership and permissions.
- Starts and enables the service at boot.
- Verifies the website is responding over HTTP.
- Combines packages, copy, variables, and service management in one workflow.
- This is the foundation for real-world automation.

8. Key Takeaways

- You can deploy a working website with a single playbook.
- Ansible connects over SSH and manages the server.
- Tasks are idempotent, so you can run the playbook multiple times.
- Always verify the service and the actual website response.
- Use --check and --diff to preview changes.
- You can limit execution to specific hosts.
- This same workflow can be extended for real applications and production environments.

Ansible for Beginners: Production Readiness: Verify, Recover, and Improve

1. Final Deployment Checklist

- Confirm the correct inventory and target hosts.
- Validate the playbook syntax before running.
- Run the playbook and check for errors.
- Run it again to confirm idempotency (changed=0).
- Verify the website is accessible over HTTP.
- Ensure secrets are protected (no_log or Ansible Vault).
- Check file permissions and ownership.
- Confirm the service is running and enabled.
- Review logs and test from another machine.
- Keep a backup or rollback plan.

2. Useful Verification Commands

```
# Check playbook syntax
ansible-playbook site.yml --syntax-check
# Run the playbook
ansible-playbook -i inventory.ini site.yml
# Run again to confirm no changes
ansible-playbook -i inventory.ini site.yml
# Check Nginx service status
systemctl status nginx
# Test the website locally on the server
curl http://localhost
# Test from your local machine
curl http://server_ip
```

3. Expected Results (Second Run)

```
PLAY RECAP *********************************************
web-01     : ok=5    changed=0    unreachable=0
             failed=0  skipped=0    rescued=0    ignored=0
```

On a second run, you should see changed=0 if nothing needs to be updated.

4. If Nginx Fails After Deployment

What to Check	Command / File	Why It Matters
Service status	systemctl status nginx	See the error message.
Configuration test	nginx -t	Find syntax errors.
Error logs	/var/log/nginx/error.log	Check detailed errors.
Configuration file	/etc/nginx/nginx.conf	Look for incorrect settings.
Port and firewall	ss -tuln | grep :80	Confirm port 80 is open.
File permissions	ls -l /var/www/html	Ensure correct ownership.
SELinux/AppArmor	getenforce	May be blocking Nginx.
Recent changes	git diff (if using Git)	Check what changed.

5. Common Problems and Fixes

Problem	Possible Cause	Fix
Nginx not starting	Invalid configuration	Run nginx -t and fix errors.
Port 80 not accessible	Firewall blocking	Allow port 80 and check security groups.
Permission denied	Wrong file ownership	Use owner: root and group: root in copy/template.
Website not updated	Wrong destination path	Check the dest path in the playbook.
Changes not applied	Using --check mode	Run without --check to apply changes.

6. Recovery Steps

1. Check the service status and logs.
2. Fix the configuration or file permissions.
3. Re-run the playbook after making corrections.
4. Use --limit to target only the affected host.
5. If needed, restore a previous working configuration from backup or Git.
6. Verify the website is accessible again.

7. Next Learning Steps

1. **Roles** – Organize your playbooks into reusable roles.
2. **Collections** – Use community and official collections.
3. **Ansible Lint** – Check for best practices and errors.
4. **CI/CD Integration** – Run playbooks in GitHub Actions or other CI/CD tools.
5. **Multi-Server Deployments** – Deploy to multiple servers and environments (dev, staging, production) in a controlled way.
6. **Monitoring and Improvements** – Add monitoring, logging, and alerts, and keep improving your automation.

8. Key Takeaways

- Always validate your playbook before running.
- Run playbooks multiple times to ensure idempotency.
- Verify the actual application (HTTP response), not just the playbook output.
- Protect secrets and never store them in plain text.
- Understand how to troubleshoot and recover quickly.
- Use proper permissions and file ownership.
- Plan for safe, controlled deployments.
- Continue learning with roles, collections, linting, and CI/CD.
- These practices make your automation reliable, secure, and ready for real-world use.